

UNIVERSIDADE FEDERAL DE UBERLÂNDIA

PPGEELT – Pós-Graduação em Engenharia Elétrica

LUCAS ALBINO MARTINS

12622EEL006

**TRABALHO 07 – RECONHECIMENTO DE DÍGITOS COM
PERCEPTRON MULTICAMADAS**

Disciplina: EL056 - Redes Neurais Artificiais

Uberlândia

2026

UNIVERSIDADE FEDERAL DE UBERLÂNDIA

PPGEELT – Pós-Graduação em Engenharia Elétrica

LUCAS ALBINO MARTINS

12622EEL006

TRABALHO 07 – RECONHECIMENTO DE DÍGITOS COM PERCEPTRON MULTICAMADAS

Disciplina: EL056 - Redes Neurais Artificiais

Trabalho apresentado ao Programa de Pós-Graduação em Engenharia Elétrica (PPGEELT) da Universidade Federal de Uberlândia como requisito parcial de avaliação da disciplina.

Disciplina: EL056 - Redes Neurais Artificiais.

Professor: Prof. Dr. Keiji Yamanaka.

Uberlândia

2026

RESUMO

Este trabalho apresenta o treinamento de um Perceptron Multicamadas (MLP) para o reconhecimento dos dígitos de 0 a 9. Foi construída uma base de 50 padrões, com os dez dígitos renderizados em cinco fontes TrueType de estilos distintos (sem serifa negrito, serifada, sem serifa fina, manuscrita e serifada condensada), reduzidos a uma grade de 12×9 pixels em representação bipolar. A rede, implementada do zero em NumPy, tem uma camada oculta de 20 neurônios com ativação tangente hiperbólica e é treinada por *backpropagation* com momento. A rede convergiu em 144 épocas e classificou corretamente todos os padrões de treino; manteve 100,0% de acurácia com até 10% dos pixels invertidos e 94,5% com 20%. No teste de generalização entre fontes (*leave-one-font-out*) a acurácia média foi de 91,2%, com o pior resultado na fonte manuscrita (62,0%). O estudo de sensibilidade mostrou que 5 neurônios ocultos são insuficientes para uma convergência confiável, enquanto 10 ou mais convergem em todas as inicializações.

Palavras-chave: redes neurais artificiais; perceptron multicamadas; *backpropagation*; reconhecimento de dígitos.

SUMÁRIO

1	Introdução	4
2	Fundamentação teórica	4
2.1	Perceptron multicamadas	4
2.2	Backpropagation	4
2.3	Termo de momento	4
3	Metodologia	5
3.1	Construção da base e pré-processamento	5
3.2	Arquitetura e hiperparâmetros	5
4	Resultados	6
4.1	Treinamento e matriz de confusão	6
4.2	Robustez a ruído	7
4.3	Generalização entre fontes	7
4.4	Sensibilidade ao número de neurônios ocultos	8
5	Discussão	8
6	Conclusão	9
	APÊNDICE A – CÓDIGO-FONTE EM PYTHON	11

1 INTRODUÇÃO

O reconhecimento de caracteres é um dos problemas clássicos de classificação de padrões e serviu de vitrine para o *backpropagation* desde a sua popularização (RUMELHART; HINTON; WILLIAMS, 1986). Em aplicações reais, como a leitura de dígitos em documentos, redes treinadas por gradiente atingem desempenho elevado quando dispõem de muitos exemplos (LECUN et al., 1998). Este trabalho trata de uma versão didática do problema: com apenas cinco exemplos por classe, um para cada fonte, avalia-se até onde uma MLP de uma camada oculta consegue (i) memorizar os padrões, (ii) tolerar ruído e (iii) reconhecer um estilo de fonte que não viu no treinamento.

Os objetivos são construir a base de dados a partir de cinco fontes, implementar a MLP e o algoritmo de treinamento sem bibliotecas de redes neurais e analisar convergência, robustez, generalização e o efeito do número de neurônios ocultos.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Perceptron multicamadas

A MLP é uma rede alimentada adiante em que uma ou mais camadas de neurônios ocultos, com ativação não linear, permitem aprender fronteiras de decisão não lineares (HAYKIN, 2009; BRAGA; CARVALHO; LUDERMIR, 2007). Com n entradas x_i , p neurônios ocultos z_j e m saídas y_k , e usando a notação de Fausett (1994), a fase direta é

$$z_j = f\left(v_{0j} + \sum_{i=1}^n x_i v_{ij}\right), \quad y_k = f\left(w_{0k} + \sum_{j=1}^p z_j w_{jk}\right), \quad (1)$$

em que v_{0j} e w_{0k} são os bias. Adotou-se a tangente hiperbólica (sigmoide bipolar), $f(x) = \tanh(x)$, cuja derivada se escreve a partir da própria saída, $f'(x) = 1 - f(x)^2$, e cuja imagem $(-1, 1)$ é compatível com entradas e alvos bipolares.

2.2 Backpropagation

O treinamento minimiza o erro quadrático $E = \frac{1}{2} \sum_k (t_k - y_k)^2$ por descida de gradiente. A regra delta generalizada (RUMELHART; HINTON; WILLIAMS, 1986) calcula os termos de erro da saída e os retropropaga para a camada oculta:

$$\delta_k = (t_k - y_k) f'(y_{in_k}), \quad \delta_j = f'(z_{in_j}) \sum_{k=1}^m \delta_k w_{jk}. \quad (2)$$

2.3 Termo de momento

Para acelerar a convergência e amortecer oscilações, cada correção recebe uma fração μ da correção anterior (HAYKIN, 2009; FAUSETT, 1994):

$$\Delta w_{jk}(t) = \alpha \delta_k z_j + \mu \Delta w_{jk}(t-1), \quad \Delta v_{ij}(t) = \alpha \delta_j x_i + \mu \Delta v_{ij}(t-1), \quad (3)$$

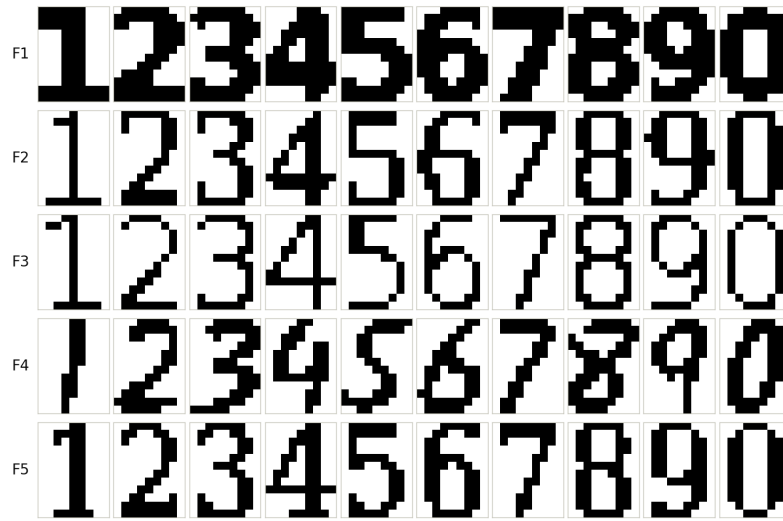
com $z_0 = x_0 = 1$ para os bias e α a taxa de aprendizagem. Com $\mu = 0$ obtém-se o gradiente descendente puro.

3 METODOLOGIA

3.1 Construção da base e pré-processamento

Os dígitos foram renderizados com a biblioteca Pillow em cinco fontes que se aproximam dos estilos do enunciado: F1 – DeJaVu Sans Bold (sem serifa negrito); F2 – Liberation Serif (serifada regular); F3 – DeJaVu Sans ExtraLight (sem serifa fina); F4 – Kalam (manuscrita); F5 – DeJaVu Serif Condensed (serifada condensada). Cada dígito foi desenhado em alta resolução, recortado pela caixa delimitadora, centralizado numa tela com a proporção da grade (preservando a proporção do caractere) e reduzido a 12×9 pixels por média de área. Pixels com fração de tinta $\geq 0,30$ recebem $+1$ e os demais -1 , resultando em vetores de 108 componentes (Figura 1). O número médio de pixels ativos por padrão variou de 29,0 (F3, traço fino) a 67,0 (F1, negrito).

Figura 1 – Base de dados: 10 dígitos $\times$ 5 fontes em grade 12×9 (preto = $+1$)



Fonte: elaborada pelo autor (2026).

As versões ruidosas invertem o sinal de uma fração exata dos 108 pixels de cada padrão (5, 10, 15 e 20%), em posições sorteadas com semente fixa.

3.2 Arquitetura e hiperparâmetros

A rede tem 108 entradas, uma camada oculta e 10 saídas com codificação bipolar *one-hot* ($+1$ na classe correta e -1 nas demais); a classe atribuída é a do neurônio de maior saída. O treinamento é padrão a padrão, em ordem aleatória a cada época, e para quando o erro quadrático total $E_{\text{total}} = \frac{1}{2} \sum_p \sum_k (t_{pk} - y_{pk})^2$ fica abaixo da tolerância ou ao atingir o limite de épocas (Tabela 1).

Tabela 1 – Hiperparâmetros da MLP

Parâmetro	Valor
Neurônios ocultos	20
Taxa de aprendizagem α	0,01
Momento μ	0,8
Tolerância de E_{total}	0,05
Máximo de épocas	5000
Pesos iniciais	$U(-0,1; 0,1)$, semente 2026

Fonte: elaborada pelo autor (2026).

A amplitude dos pesos iniciais foi reduzida em relação ao valor usual de $\pm 0,5$: com 108 entradas bipolares, essa amplitude leva a entrada líquida dos neurônios ocultos a valores que saturam a $\tanh$ ($f' \approx 0$), e num ensaio preliminar a rede não convergiu. Toda a implementação usa apenas NumPy (HARRIS et al., 2020) (Apêndice A).

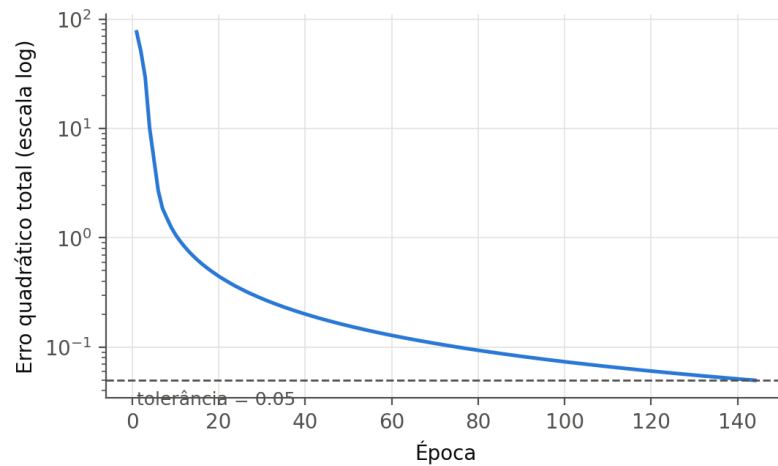
Nos experimentos de ruído, cada nível foi avaliado em 30 repetições sobre os 50 padrões. No *leave-one-font-out* e no estudo de sensibilidade, cada configuração foi treinada com 5 inicializações (sementes 2026 a 2030).

4 RESULTADOS

4.1 Treinamento e matriz de confusão

O erro quadrático total caiu de 76,24 na primeira época para 0,0498 na época 144, quando o critério de parada foi atingido (Figura 2). Sem o termo de momento, a mesma rede, com a mesma inicialização, precisou de 663 épocas. Nos dois casos a acurácia de treino foi de 100,0%, e a matriz de confusão de treino é diagonal (Figura 4, à esquerda).

Figura 2 – Erro quadrático total por época (20 neurônios ocultos, com momento)

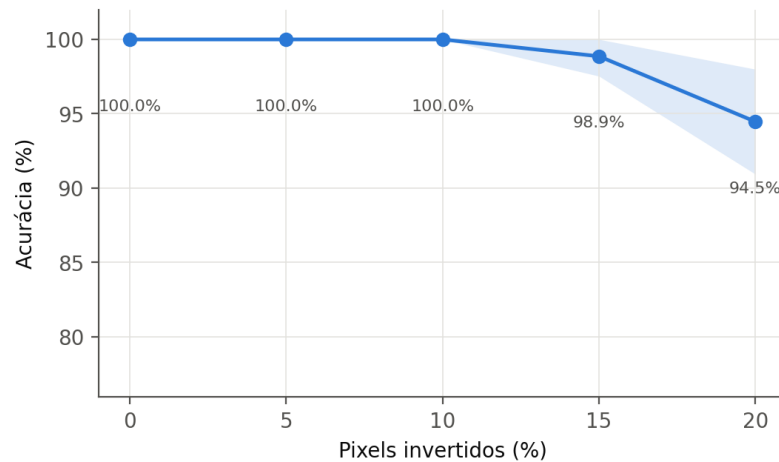


Fonte: elaborada pelo autor (2026).

4.2 Robustez a ruído

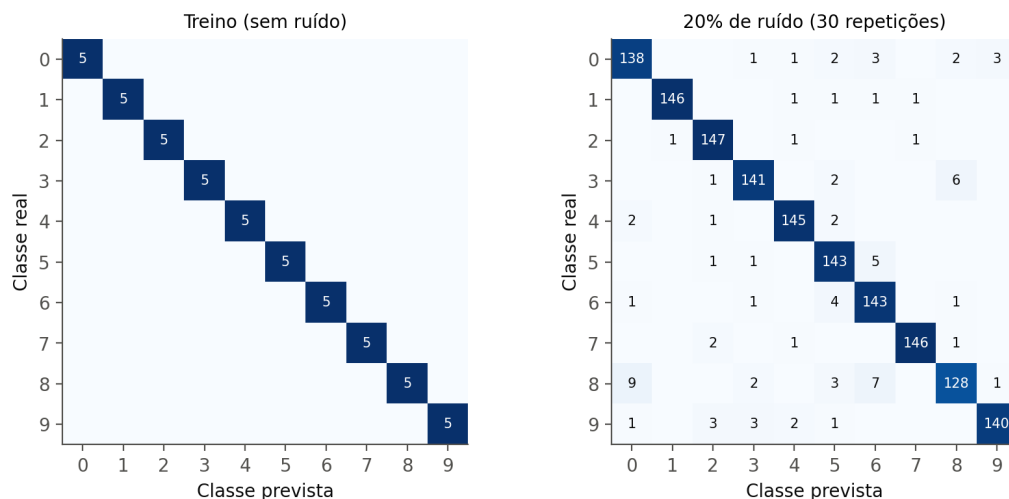
A acurácia permaneceu em 100,0% com 5% e 10% de pixels invertidos, caiu para 98,9% (desvio padrão de 1,3 ponto) com 15% e para 94,5% (desvio de 3,5) com 20% (Figura 3). Com 10% de ruído, a acurácia foi de 100,0% em cada uma das cinco fontes. A matriz acumulada sob 20% de ruído (Figura 4, à direita) mostra que o dígito 8 é o mais afetado: 128 acertos em 150, com confusões principalmente para 0 (9 casos) e 6 (7 casos); em seguida aparece a troca de 3 por 8 (6 casos).

Figura 3 – Acurácia média em função do nível de ruído (faixa: ± 1 desvio padrão, 30 repetições)



Fonte: elaborada pelo autor (2026).

Figura 4 – Matrizes de confusão: treino sem ruído e com 20% de ruído (30 repetições acumuladas)



Fonte: elaborada pelo autor (2026).

4.3 Generalização entre fontes

A Tabela 2 resume o teste *leave-one-font-out*. As três fontes serifadas ou finas foram reconhecidas integralmente quando deixadas de fora; a negrito teve 94,0% e a manuscrita, 62,0%.

A média geral foi de 91,2%. Os treinos com 40 padrões convergiram, em média, entre 137,6 e 144,0 épocas, conforme a fonte excluída.

Tabela 2 – Acurácia na fonte excluída do treino (média $\pm$ desvio de 5 inicializações)

Fonte de teste	Erros (semente 2026)	Acurácia (%)
F1 – sem serifa negrito	—	94,0 $\pm$ 8,0
F2 – serifada regular	—	100,0 $\pm$ 0,0
F3 – sem serifa fina	—	100,0 $\pm$ 0,0
F4 – manuscrita	4 $\rightarrow$ 9, 5 $\rightarrow$ 7, 6 $\rightarrow$ 8	62,0 $\pm$ 7,5
F5 – serifada condensada	—	100,0 $\pm$ 0,0
Média geral		91,2

Fonte: elaborada pelo autor (2026).

4.4 Sensibilidade ao número de neurônios ocultos

Com 5 neurônios ocultos, apenas 2 das 5 inicializações atingiram a tolerância antes de 5000 épocas; a acurácia média de treino foi de 91,6% e, com 10% de ruído, de 85,0%. A partir de 10 neurônios todas as inicializações convergiram, e o número de épocas caiu à medida que a camada oculta cresceu (Tabela 3).

Tabela 3 – Efeito do número de neurônios ocultos (média de 5 inicializações)

Ocultos	Épocas	Convergiram	Acurácia com 10% de ruído (%)
5	3469,0	2/5	85,0
10	330,6	5/5	99,2
20	144,6	5/5	99,9
40	81,4	5/5	99,9

Fonte: elaborada pelo autor (2026).

5 DISCUSSÃO

A rede memoriza com facilidade os 50 padrões, e o momento reduziu as épocas de treinamento em cerca de 4,6 vezes (663 contra 144), o que ilustra seu papel de acumular a direção consistente do gradiente (HAYKIN, 2009). A robustez a ruído é alta porque, em 108 pixels, inverter 10% ainda deixa o padrão mais próximo do seu protótipo do que dos demais; as falhas com 20% de ruído concentram-se em pares que compartilham traços na grade 12×9 (8, 0 e 6; 3 e 8; 5 e 6).

A generalização entre fontes revela o limite de uma base tão pequena. As fontes F2, F3 e F5 têm formas parecidas depois da redução para 12×9 , de modo que qualquer uma delas é bem representada pelas demais. A negrito ocupa em média 67 pixels, contra cerca de 40 nas outras, e por isso depende mais da inicialização (desvio de 8 pontos). A manuscrita é a mais

distante: o 4 aberto, o 5 e o 6 inclinados e o 1 sem base não têm correspondentes no treino, e com um único exemplo por fonte a rede não tem como aprender essa variação. Em problemas reais, isso se resolve com muito mais exemplos por classe e, em geral, com arquiteturas que exploram a estrutura espacial da imagem (LECUN et al., 1998).

Quanto ao tamanho da camada oculta, 5 neurônios formam um gargalo: a rede fica presa em mínimos em que algumas classes não se separam. De 10 em diante, mais neurônios aceleram o treino, mas a robustez com 10% de ruído praticamente não muda (99,2% a 99,9%). Os 20 neurônios adotados equilibram convergência rápida e tamanho da rede.

6 CONCLUSÃO

Foi construída uma base de 50 dígitos em cinco fontes e treinada uma MLP implementada do zero. A rede convergiu em 144 épocas, reconheceu todos os padrões de treino e manteve 94,5% de acurácia com 20% dos pixels invertidos. A principal limitação apareceu no teste entre fontes: 91,2% de acurácia média, com apenas 62,0% na fonte manuscrita, o que mostra que cinco exemplos por classe bastam para memorizar e tolerar ruído, mas não para generalizar para um estilo de escrita diferente. Como continuidade, seria possível ampliar a base com mais fontes e com pequenas translações e rotações dos dígitos existentes.

REFERÊNCIAS

BRAGA, A. de P.; CARVALHO, A. P. d. L. F. d.; LUDERMIR, T. B. *Redes Neurais Artificiais: Teoria e Aplicações*. 2. ed. Rio de Janeiro: LTC, 2007.

FAUSETT, L. *Fundamentals of Neural Networks: Architectures, Algorithms, and Applications*. Englewood Cliffs, NJ: Prentice-Hall, 1994.

HARRIS, C. R. et al. Array programming with NumPy. *Nature*, v. 585, n. 7825, p. 357–362, 2020.

HAYKIN, S. *Neural Networks and Learning Machines*. 3. ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2009.

LECUN, Y. et al. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, v. 86, n. 11, p. 2278–2324, 1998.

RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*, v. 323, n. 6088, p. 533–536, 1986.

APÊNDICE A – CÓDIGO-FONTE EM PYTHON

Arquivo trabalho07_digitos.py, executado com `python trabalho07_digitos.py`.

```

1  # -*- coding: utf-8 -*-
2  """
3  Trabalho 07 - Reconhecimento de Dígitos com Perceptron Multicamadas (MLP)
4
5  Disciplina : EL056 - Redes Neurais Artificiais (PPGEELT/UFU)
6  Professor  : Prof. Dr. Keiji Yamanaka
7  Aluno      : Lucas Albino Martins (12622EEL006)
8
9  O programa:
10  1. constrói uma base de dados com os dígitos 0-9 renderizados em cinco
11     fontes TrueType diferentes (50 padrões), em representação bipolar;
12  2. gera versões ruidosas da base (inversão aleatória de pixels);
13  3. implementa do zero, em NumPy, uma MLP com uma camada oculta, ativação
14     tangente hiperbólica e treinamento por backpropagation com momento;
15  4. executa os experimentos pedidos (curva de erro, matriz de confusão,
16     robustez a ruído, generalização entre fontes e sensibilidade ao número
17     de neurônios ocultos);
18  5. salva figuras e todos os resultados numéricos em figuras/.
19
20  Uso:
21      python trabalho07_digitos.py                # executa tudo
22      python trabalho07_digitos.py --saida out    # muda a pasta de saída
23  """
24
25  import argparse
26  import json
27  import os
28  import sys
29  import time
30  import warnings
31
32  import numpy as np
33  from PIL import Image, ImageDraw, ImageFont
34
35  import matplotlib
36  matplotlib.use("Agg")                # gera figuras sem abrir janelas
37  import matplotlib.pyplot as plt
38
39  # =====
40  # CONFIGURAÇÃO GERAL (altere aqui)
41  # =====
42
43  PASTA_SCRIPT = os.path.dirname(os.path.abspath(__file__)) if "__file__" in globals() else os.getcwd()
44
45  # ---- Fontes: (rótulo, [arquivos candidatos em ordem de preferência]) ----
46  # O primeiro arquivo encontrado é usado. Se o preferido não existir, o
47  # programa avisa e usa a próxima alternativa da lista. Para usar outra
48  # fonte basta colocar o caminho completo (ou só o nome do .ttf) no início
49  # da lista correspondente.
50  FONTES = [
51      ("Sem serifa negrito (DejaVu Sans Bold)",
52       ["DejaVuSans-Bold.ttf", "LiberationSans-Bold.ttf", "arialbd.ttf", "Arial Bold.ttf"]),
53      ("Serifada regular (Liberation Serif)",
54       ["LiberationSerif-Regular.ttf", "times.ttf", "Times New Roman.ttf", "DejaVuSerif.ttf"]),
55      ("Sem serifa fina (DejaVu Sans ExtraLight)",
56       ["DejaVuSans-ExtraLight.ttf", "Poppins-Light.ttf", "segoeul.ttf", "LiberationSans-Regular.ttf"]),
57      ("Manuscrita/cursiva (Kalam)",
58       ["Kalam-Regular.ttf", "segoepr.ttf", "LiberationSerif-Italic.ttf", "DejaVuSerif-Italic.ttf"]),
59      ("Serifada condensada (DejaVu Serif Condensed)",
60       ["DejaVuSerifCondensed.ttf", "DejaVuSerif.ttf", "georgia.ttf", "LiberationSerif-Regular.ttf"]),
61  ]
62
63  # Pastas onde os arquivos de fonte são procurados (a pasta fontes/ do
64  # projeto vem primeiro, garantindo reprodutibilidade em qualquer sistema).
65  PASTAS_FONTES = [
66      os.path.join(PASTA_SCRIPT, "fontes"),
67      "/usr/share/fonts",
68      "/usr/local/share/fonts",
69      os.path.expanduser("~/fonts"),

```

```

70 os.path.expanduser("~/local/share/fonts"),
71 "/Library/Fonts",
72 "/System/Library/Fonts",
73 os.path.expanduser("~/Library/Fonts"),
74 "C:/Windows/Fonts",
75 ]
76
77 DIGITOS = ["1", "2", "3", "4", "5", "6", "7", "8", "9", "0"] # ordem do enunciado
78 LINHAS, COLUNAS = 12, 9 # grade de pixels de cada padrão (12 x 9 = 108 entradas)
79 TAMANHO_FONTE = 200 # tamanho (pt) usado na renderização em alta resolução
80 LIMIAR = 0.30 # fração mínima de tinta para o pixel virar +1
81
82 # ---- Hiperparâmetros da MLP ----
83 N_OCULTOS = 20 # neurônios na camada oculta
84 ALFA = 0.01 # taxa de aprendizagem
85 MOMENTO = 0.8 # termo de momento (0 desativa)
86 TOLERANCIA = 0.05 # erro quadrático total de parada
87 MAX_EPOCAS = 5000 # limite de épocas
88 AMPLITUDE_PESOS = 0.1 # pesos iniciais ~  $U(-A, +A)$ ; com 108 entradas,
89 #  $A = 0.5$  satura a tanh logo no início do treino
90 SEMENTE = 2026 # semente global (reprodutibilidade)
91
92 # ---- Experimentos ----
93 NIVEIS_RUIDO = [0.0, 0.05, 0.10, 0.15, 0.20] # fração de pixels invertidos
94 REPETICOES_RUIDO = 30 # repetições por nível
95 OCULTOS_SENSIBILIDADE = [5, 10, 20, 40] # estudo de sensibilidade
96 SEMENTES_SENSIBILIDADE = 5 # inicializações por configuração
97 SEMENTES_LOFO = 5 # inicializações no leave-one-font-out
98
99 # Cores das figuras (tons sóbrios, adequados à impressão)
100 COR_PRINCIPAL = "#2a78d6"
101 COR_SECUNDARIA = "#eb6834"
102 COR_TEXTO = "#52514e"
103
104
105 # =====
106 # 1. CONSTRUÇÃO DA BASE DE DADOS
107 # =====
108
109 def _indexar_pastas(pastas):
110     """Percorre as pastas de fontes uma única vez e devolve {nome_minúsculo: caminho}."""
111     indice = {}
112     for pasta in pastas:
113         if not os.path.isdir(pasta):
114             continue
115         for raiz, _, arquivos in os.walk(pasta):
116             for arq in arquivos:
117                 if arq.lower().endswith((".ttf", ".otf")):
118                     indice.setdefault(arq.lower(), os.path.join(raiz, arq))
119     return indice
120
121
122 def localizar_fontes(fontes=FONTES, pastas=PASTAS_FONTES, tamanho=TAMANHO_FONTE):
123     """Carrega as fontes configuradas, com aviso e alternativa quando faltarem.
124
125     Retorna uma lista de tuplas (rótulo, caminho_usado, objeto ImageFont).
126     """
127     indice = _indexar_pastas(pastas)
128     carregadas = []
129     for rotulo, candidatos in fontes:
130         caminho = None
131         for i, cand in enumerate(candidatos):
132             achado = cand if os.path.isfile(cand) else indice.get(os.path.basename(cand).lower())
133             if achado:
134                 caminho = achado
135                 if i > 0:
136                     warnings.warn(f"Fonte preferida '{candidatos[0]}' não encontrada para "
137                                   f"'{rotulo}'; usando alternativa '{cand}'.")
138                 break
139         if caminho is None:
140             warnings.warn(f"Não nenhuma fonte encontrada para '{rotulo}'; usando a fonte "
141                           f"padrão do Pillow (o resultado pode ficar diferente).")
142         fonte = ImageFont.load_default(size=tamanho)

```

```

143     caminho = "<padrão do Pillow>"
144     else:
145         fonte = ImageFont.truetype(caminho, tamanho)
146         carregadas.append((rotulo, caminho, fonte))
147     return carregadas
148
149
150 def renderizar_digito(fonte, digito, linhas=LINHAS, colunas=COLUNAS, limiar=LIMIAR):
151     """Renderiza um dígito e o converte em uma matriz bipolar linhas x colunas.
152
153     Passos: desenho em alta resolução -> recorte pela caixa delimitadora ->
154     centralização numa tela com a proporção da grade (mantendo a proporção
155     do dígito) -> redução por média de área -> binarização -> bipolar.
156     """
157     lado = 3 * TAMANHO_FONTE
158     img = Image.new("L", (lado, lado), 0) # fundo 0, tinta 255
159     ImageDraw.Draw(img).text((lado // 3, lado // 6), digito, font=fonte, fill=255)
160     caixa = img.getbbox() # caixa delimitadora da tinta
161     recorte = img.crop(caixa)
162     w, h = recorte.size
163
164     # Tela com a mesma proporção da grade, grande o bastante para o dígito
165     proporcao = colunas / linhas
166     alt = max(h, int(np.ceil(w / proporcao)))
167     larg = int(round(alt * proporcao))
168     tela = Image.new("L", (larg, alt), 0)
169     tela.paste(recorte, ((larg - w) // 2, (alt - h) // 2))
170
171     # Redução por média de área (BOX): cada pixel final = fração de tinta
172     reduzida = np.asarray(tela.resize((colunas, linhas), Image.BOX), dtype=float) / 255.0
173     binaria = reduzida >= limiar
174     return np.where(binaria, 1, -1).astype(np.int8)
175
176
177 def construir_base(fontes_carregadas, digitos=DIGITOS):
178     """Monta a base completa: X (50 x 108) bipolar, rótulos (0-9) e índice da fonte."""
179     X, y, id_fonte = [], [], []
180     for f, (_, _, fonte) in enumerate(fontes_carregadas):
181         for d in digitos:
182             X.append(renderizar_digito(fonte, d).ravel())
183             y.append(int(d))
184             id_fonte.append(f)
185     return np.array(X, dtype=float), np.array(y), np.array(id_fonte)
186
187
188 def adicionar_ruido(X, fracao, rng):
189     """Inverte o sinal de uma fração exata dos pixels de cada padrão (ruído)."""
190     Xr = X.copy()
191     n = X.shape[1]
192     k = int(round(fracao * n))
193     for i in range(X.shape[0]):
194         idx = rng.choice(n, size=k, replace=False)
195         Xr[i, idx] *= -1
196     return Xr
197
198
199 def codificar_alvos(y, n_classes=10):
200     """Codificação bipolar one-hot: +1 na classe correta, -1 nas demais."""
201     T = -np.ones((len(y), n_classes))
202     T[np.arange(len(y)), y] = 1.0
203     return T
204
205
206 # =====
207 # 2. REDE MLP (implementação própria em NumPy)
208 # =====
209
210 class MLP:
211     """Perceptron multicamadas com uma camada oculta e ativação tanh.
212
213     V : pesos entrada -> oculta, forma (n_entradas + 1, n_ocultos); linha 0 = bias
214     W : pesos oculta -> saída, forma (n_ocultos + 1, n_saídas); linha 0 = bias
215     """

```

```

216
217 def __init__(self, n_entradas, n_ocultos, n_saidas, semente=SEMENTE,
218             amplitude=AMPLITUDE_PESOS):
219     rng = np.random.default_rng(semente)
220     self.V = rng.uniform(-amplitude, amplitude, (n_entradas + 1, n_ocultos))
221     self.W = rng.uniform(-amplitude, amplitude, (n_ocultos + 1, n_saidas))
222     self.rng = rng
223
224 @staticmethod
225 def f(x):
226     """Função de ativação: tangente hiperbólica (sigmoide bipolar)."""
227     return np.tanh(x)
228
229 @staticmethod
230 def df(fx):
231     """Derivada da tanh escrita em função da própria saída: 1 - f(x)^2."""
232     return 1.0 - fx ** 2
233
234 def propagar(self, X):
235     """Fase direta. Aceita um padrão (vetor) ou vários (matriz)."""
236     Z = self.f(X @ self.V[1:] + self.V[0]) # saídas da camada oculta
237     Y = self.f(Z @ self.W[1:] + self.W[0]) # saídas da rede
238     return Z, Y
239
240 def erro_total(self, X, T):
241     """Erro quadrático total:  $E = 1/2 * \sum (t_{pk} - y_{pk})^2$ ."""
242     _, Y = self.propagar(X)
243     return 0.5 * float(np.sum((T - Y) ** 2))
244
245 def treinar(self, X, T, alfa=ALFA, momento=MOMENTO, tolerancia=TOLERANCIA,
246            max_epocas=MAX_EPOCAS, verboso=False):
247     """Backpropagation padrão a padrão (online), com momento.
248
249     Parada:  $E < \text{tolerância}$  ou número máximo de épocas.
250     Retorna a lista com o erro quadrático total ao fim de cada época.
251     """
252     dV_ant = np.zeros_like(self.V)
253     dW_ant = np.zeros_like(self.W)
254     historico = []
255     for epoca in range(1, max_epocas + 1):
256         for p in self.rng.permutation(len(X)): # ordem aleatória a cada época
257             x, t = X[p], T[p]
258             # --- fase direta ---
259             z, yv = self.propagar(x)
260             # --- retropropagação do erro ---
261             delta_k = (t - yv) * self.df(yv) # termos de erro da saída
262             delta_j = (self.W[1:] @ delta_k) * self.df(z) # termos de erro da oculta
263             # --- correções (gradiente + momento) ---
264             dW = alfa * np.outer(np.concatenate([[1.0], z]), delta_k) + momento * dW_ant
265             dW = alfa * np.outer(np.concatenate([[1.0], x]), delta_j) + momento * dV_ant
266             self.W += dW
267             self.V += dV
268             dW_ant, dV_ant = dW, dV
269             E = self.erro_total(X, T)
270             historico.append(E)
271             if verboso and (epoca % 100 == 0 or E < tolerancia):
272                 print(f"  época {epoca:5d}  E = {E:.5f}")
273             if E < tolerancia:
274                 break
275     return historico
276
277 def classificar(self, X):
278     """Classe = índice do neurônio de saída com maior valor."""
279     _, Y = self.propagar(X)
280     return np.argmax(np.atleast_2d(Y), axis=1)
281
282
283 def acuracia(rede, X, y):
284     return float(np.mean(rede.classificar(X) == y))
285
286
287 def treinar_rede(X, y, n_ocultos=N_OCULTOS, semente=SEMENTE, verboso=False):
288     """Cria e treina uma MLP; devolve (rede, histórico de erro)."""

```

```

289 rede = MLP(X.shape[1], n_ocultos, 10, semente=semente)
290 hist = rede.treinar(X, codificar_alvos(y), verboso=verboso)
291 return rede, hist
292
293
294 # =====
295 # 3. EXPERIMENTOS
296 # =====
297
298 def matriz_confusao(y_real, y_prev, n=10):
299     M = np.zeros((n, n), dtype=int)
300     for r, p in zip(y_real, y_prev):
301         M[r, p] += 1
302     return M
303
304
305 def teste_ruido(rede, X, y, niveis=NIVEIS_RUIDO, repeticoes=REPETICOES_RUIDO, semente=SEMENTE):
306     """Acurácia média (e desvio) para cada nível de ruído."""
307     rng = np.random.default_rng(semente + 1)
308     medias, desvios = [], []
309     for nivel in niveis:
310         accs = [acuracia(rede, adicionar_ruido(X, nivel, rng), y) for _ in range(repeticoes)]
311         medias.append(float(np.mean(accs)))
312         desvios.append(float(np.std(accs)))
313     return medias, desvios
314
315
316 def leave_one_font_out(X, y, id_fonte, rotulos, n_sementes=SEMENTES_LOFO):
317     """Treina com 4 fontes e testa na quinta, para as 5 combinações."""
318     resultados = []
319     for f in range(len(rotulos)):
320         treino, teste = id_fonte != f, id_fonte == f
321         accs, epocas, erros_sem0 = [], [], None
322         for s in range(n_sementes):
323             rede, hist = treinar_rede(X[treino], y[treino], semente=SEMENTE + s)
324             prev = rede.classificar(X[teste])
325             accs.append(float(np.mean(prev == y[teste])))
326             epocas.append(len(hist))
327             if s == 0:
328                 erros_sem0 = [f"{r}->{p}" for r, p in zip(y[teste], prev) if r != p]
329             resultados.append({
330                 "fonte_teste": rotulos[f],
331                 "acuracia_media": float(np.mean(accs)),
332                 "acuracia_desvio": float(np.std(accs)),
333                 "acuracias": accs,
334                 "epocas_media": float(np.mean(epocas)),
335                 "erros_semente_principal": erros_sem0,
336             })
337     return resultados
338
339
340 def sensibilidade_ocultos(X, y, lista=OCULTOS_SENSIBILIDADE, n_sementes=SEMENTES_SENSIBILIDADE):
341     """Varia o n° de neurônios ocultos: épocas até convergir e acurácia com 10% de ruído."""
342     saida = []
343     for n in lista:
344         epocas, convergiu, acc_treino, acc_r10 = [], [], [], []
345         for s in range(n_sementes):
346             rede, hist = treinar_rede(X, y, n_ocultos=n, semente=SEMENTE + s)
347             epocas.append(len(hist))
348             convergiu.append(hist[-1] < TOLERANCIA)
349             acc_treino.append(acuracia(rede, X, y))
350             m, _ = teste_ruido(rede, X, y, niveis=[0.10], semente=SEMENTE + 100 + s)
351             acc_r10.append(m[0])
352         saida.append({
353             "n_ocultos": n,
354             "epocas_media": float(np.mean(epocas)),
355             "epocas_desvio": float(np.std(epocas)),
356             "epocas": epocas,
357             "convergiu": int(sum(convergiu)),
358             "acuracia_treino_media": float(np.mean(acc_treino)),
359             "acuracia_ruido10_media": float(np.mean(acc_r10)),
360             "acuracia_ruido10_desvio": float(np.std(acc_r10)),
361         })

```

```

362     return saida
363
364
365 # =====
366 # 4. GRÁFICOS
367 # =====
368
369 def _estilo():
370     plt.rcParams.update({
371         "font.size": 10, "axes.edgecolor": COR_TEXTO, "axes.labelcolor": "#0b0b0b",
372         "xtick.color": COR_TEXTO, "ytick.color": COR_TEXTO,
373         "axes.spines.top": False, "axes.spines.right": False,
374         "axes.grid": True, "grid.color": "#e4e3df", "grid.linewidth": 0.6,
375         "savefig.dpi": 200, "savefig.bbox": "tight",
376     })
377
378
379 def fig_base(X, rotulos, caminho, titulo=None):
380     """Grade 5 x 10 com todos os padrões binarizados (preto = +1)."""
381     n_f = len(rotulos)
382     fig, eixos = plt.subplots(n_f, len(DIGITOS), figsize=(len(DIGITOS) * 0.75, n_f * 1.0))
383     for i in range(n_f):
384         for j in range(len(DIGITOS)):
385             ax = eixos[i, j]
386             ax.imshow(X[i * len(DIGITOS) + j].reshape(LINHAS, COLUNAS), cmap="gray_r",
387                     vmin=-1, vmax=1, interpolation="nearest")
388             ax.set_xticks([]); ax.set_yticks([]); ax.grid(False)
389             for s in ax.spines.values():
390                 s.set_visible(True); s.set_color("#c3c2b7"); s.set_linewidth(0.5)
391             eixos[i, 0].set_ylabel(f"F{i + 1}", rotation=0, labelpad=12, va="center")
392     if titulo:
393         fig.suptitle(titulo, fontsize=10)
394     fig.tight_layout(pad=0.3)
395     fig.savefig(caminho)
396     plt.close(fig)
397
398
399 def fig_curva_erro(hist, caminho, tolerancia=TOLERANCIA):
400     fig, ax = plt.subplots(figsize=(6, 3.4))
401     ax.plot(np.arange(1, len(hist) + 1), hist, color=COR_PRINCIPAL, lw=1.8)
402     ax.axhline(tolerancia, color=COR_TEXTO, lw=1, ls="--")
403     ax.text(1, tolerancia * 0.8, f"tolerância = {tolerancia}", ha="left",
404           va="top", color=COR_TEXTO, fontsize=9)
405     ax.set_yscale("log")
406     ax.set_xlabel("Época")
407     ax.set_ylabel("Erro quadrático total (escala log)")
408     fig.savefig(caminho)
409     plt.close(fig)
410
411
412 def fig_confusao(matrizes, titulos, caminho):
413     """Uma ou mais matrizes de confusão lado a lado (linhas = classe real)."""
414     fig, eixos = plt.subplots(1, len(matrizes), figsize=(4.3 * len(matrizes), 4.0))
415     eixos = np.atleast_1d(eixos)
416     for ax, M, tit in zip(eixos, matrizes, titulos):
417         ax.imshow(M, cmap="Blues", vmin=0, vmax=max(1, M.max()))
418         ax.grid(False)
419         ax.set_xticks(range(10)); ax.set_yticks(range(10))
420         ax.set_xlabel("Classe prevista"); ax.set_ylabel("Classe real")
421         ax.set_title(tit, fontsize=10)
422         for i in range(10):
423             for j in range(10):
424                 if M[i, j]:
425                     ax.text(j, i, M[i, j], ha="center", va="center", fontsize=7.5,
426                           color="white" if M[i, j] > M.max() / 2 else "#0b0b0b")
427     fig.tight_layout()
428     fig.savefig(caminho)
429     plt.close(fig)
430
431
432 def fig_ruido(niveis, medias, desvios, caminho):
433     fig, ax = plt.subplots(figsize=(6, 3.4))
434     pct = [100 * n for n in niveis]

```



```

508
509 # Comparação: mesmo treino sem o termo de momento
510 rede_sm = MLP(X.shape[1], N_OCULTOS, 10, semente=SEMENTE)
511 hist_sm = rede_sm.treinar(X, codificar_alvos(y), momento=0.0)
512 print(f"    sem momento: {len(hist_sm)} épocas (com momento: {len(hist)})")
513
514 # ---- 3.4 Leave-one-font-out ----
515 print("[4] Generalização entre fontes (leave-one-font-out) ...")
516 lofo = leave_one_font_out(X, y, id_fonte, rotulos)
517 for r in lofo:
518     print(f"    teste em {r['fonte_teste'][:45s]} acurácia = {100 * r['acuracia_media']:.1f}%")
519
520 # ---- 3.5 Sensibilidade ao n° de neurônios ocultos ----
521 print("[5] Sensibilidade ao número de neurônios ocultos ...")
522 sens = sensibilidade_ocultos(X, y)
523 for r in sens:
524     print(f"    {r['n_ocultos']:3d} ocultos: {r['epocas_media']:7.1f} épocas, "
525           f"    {r['acuracia_10%_ruído']:10.1f}%")
526
527 # ---- Salvar resultados ----
528 resultados = {
529     "configuracao": {
530         "fontes": [{"rotulo": r, "arquivo": os.path.basename(c)} for r, c, _ in fontes],
531         "grade": [LINHAS, COLUNAS], "n_entradas": int(X.shape[1]), "n_padroes": int(X.shape[0]),
532         "limiar_binarizacao": LIMIAR, "n_ocultos": N_OCULTOS, "alfa": ALFA,
533         "momento": MOMENTO, "tolerancia": TOLERANCIA, "max_epocas": MAX_EPOCAS,
534         "amplitude_pesos": AMPLITUDE_PESOS, "semente": SEMENTE,
535         "niveis_ruído": NIVEIS_RUIDO, "repeticoes_ruído": REPETICOES_RUIDO,
536     },
537     "base": {
538         "pixels_ativos_media_por_fonte": [float(np.mean(np.sum(X[id_fonte == f] > 0, axis=1)))
539                                           for f in range(len(rotulos))],
540     },
541     "treino": {
542         "epocas": len(hist), "erro_inicial": hist[0], "erro_final": hist[-1],
543         "convergiu": bool(hist[-1] < TOLERANCIA), "acuracia": acc_treino,
544         "epocas_sem_momento": len(hist_sm), "acuracia_sem_momento": acuracia(rede_sm, X, y),
545         "matriz_confusao": M.tolist(),
546         "matriz_confusao_ruído20": M20.tolist(),
547     },
548     "ruído": {"niveis": NIVEIS_RUIDO, "acuracia_media": medias, "acuracia_desvio": desvios,
549              "acuracia_por_fonte_ruído10": acc_fonte_r10},
550     "leave_one_font_out": lofo,
551     "leave_one_font_out_media_geral": float(np.mean([r["acuracia_media"] for r in lofo])),
552     "sensibilidade_ocultos": sens,
553     "tempo_execucao_s": round(time.time() - t0, 1),
554 }
555 with open(os.path.join(pasta, "resultados.json"), "w", encoding="utf-8") as fp:
556     json.dump(resultados, fp, ensure_ascii=False, indent=2)
557 print(f"Concluído em {resultados['tempo_execucao_s']} s. Saídas em: {pasta}")
558 return resultados
559
560
561 if __name__ == "__main__":
562     parser = argparse.ArgumentParser(description="Trabalho 07 - Reconhecimento de dígitos com MLP")
563     parser.add_argument("--saida", default=None, help="pasta de saída (padrão: ./figuras)")
564     args = parser.parse_args()
565     main(args.saida)

```